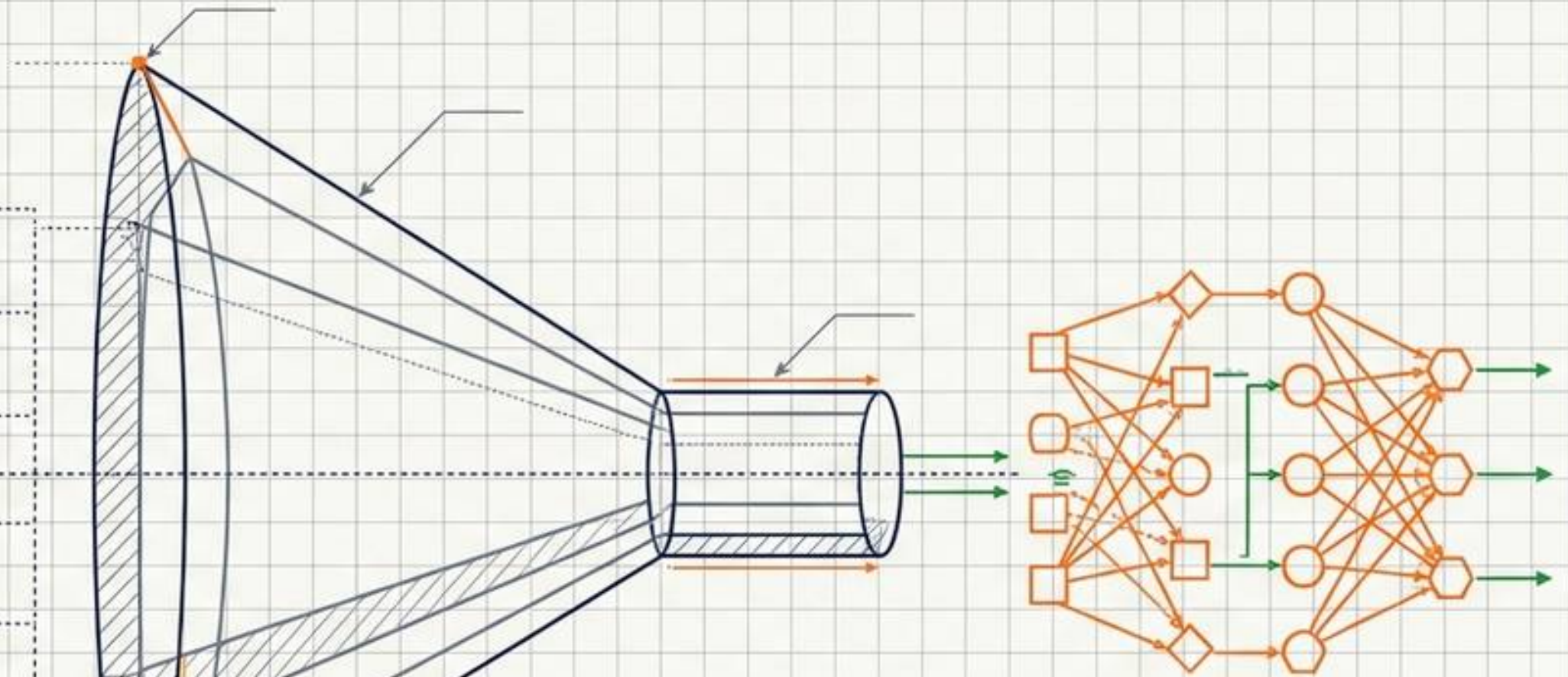
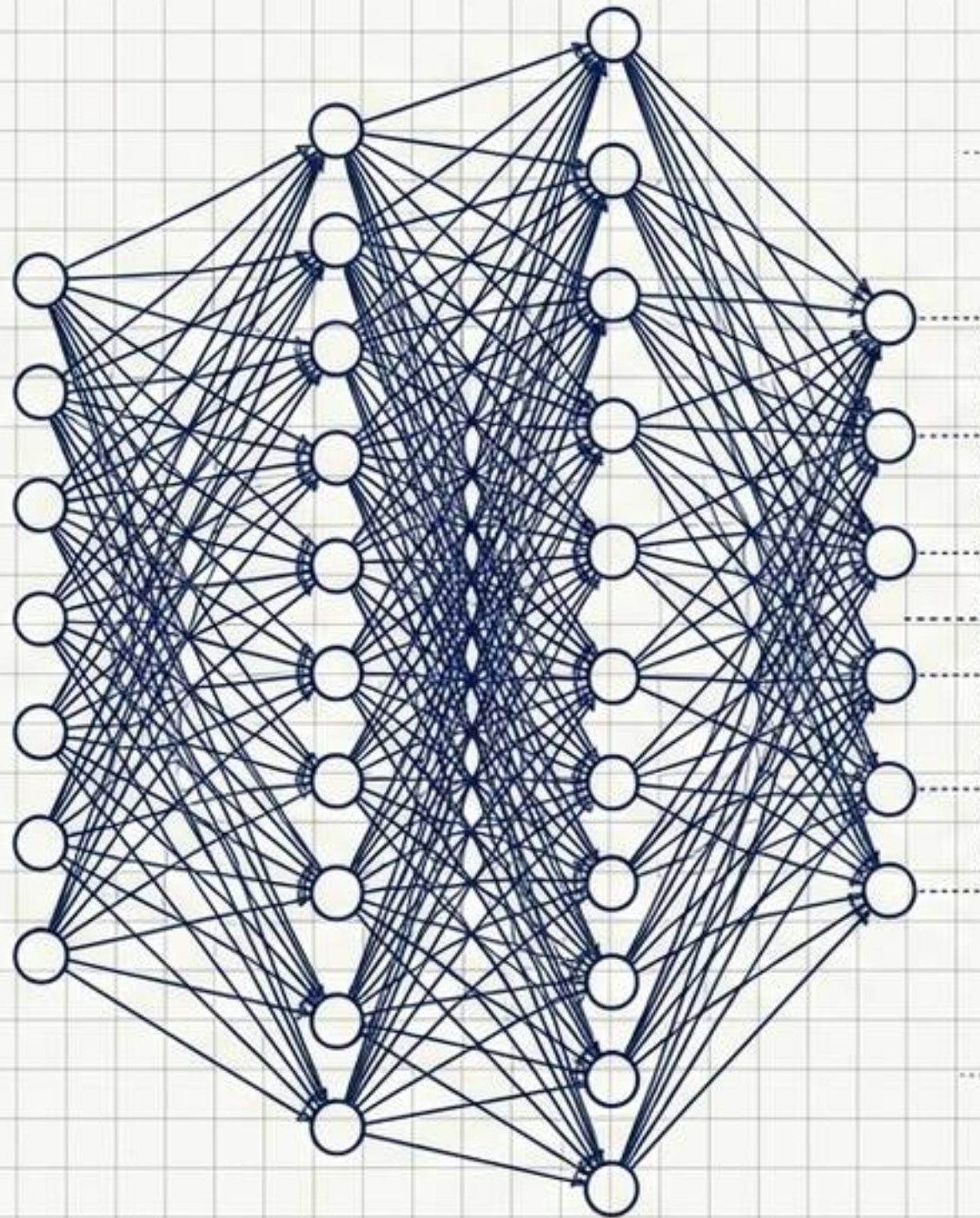




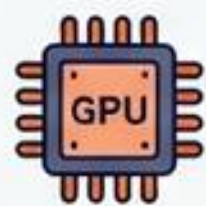
Optimización de Modelos en Machine Learning

Cuantificación, Poda de Redes y el paso hacia Producción

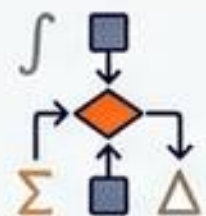
El modelo entrenado NO está listo para Producción

Entorno de Entrenamiento

(Prioridad: Aprender)



Hardware: GPU múltiple, memoria masiva, tiempo prolongado.



Proceso: Backpropagation, cálculo y ajuste de gradientes.



Métrica Clave: Accuracy y Generalización.



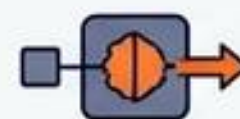
**SHIFT
GEARS**

Entorno de Producción

(Prioridad: Inferencia Eficiente)



Hardware: Servidores web, Smartphones, Dispositivos Edge.



Proceso: Nueva entrada → Modelo → Predicción (Forward pass).



Métrica Clave: Velocidad, Tamaño, Consumo de recursos.



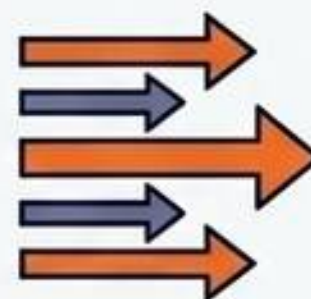
El Nuevo Dashboard de Evaluación: Más allá del Accuracy



Latencia

Tiempo exacto para procesar una entrada única. Crítica en interactividad.

Ejemplo: 120 ms por imagen



Throughput

Volumen de entradas procesadas por unidad de tiempo. Crítico en Batch.

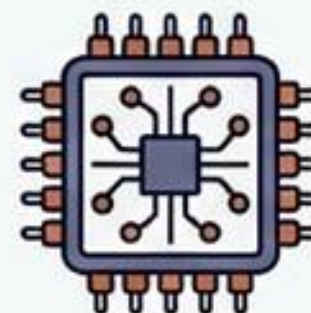
Ejemplo: 50 imágenes / segundo



Tamaño del Modelo

Espacio físico de almacenamiento en disco (parámetros × representación).

Ejemplo: 150 MB



Consumo de Memoria

RAM especializada requerida durante la inferencia (activaciones, buffers).

Ejemplo: Dependiente de arquitectura

El peso de los parámetros: El costo de FP32

Durante el entrenamiento, los pesos se almacenan en FP32 para obtener máxima precisión decimal. En la inferencia, esta precisión es excesiva y costosa.

FP32 (Floating Point de 32 bits)



$$\begin{array}{r} 10.000.000 \text{ de parámetros} \\ \times \quad 4 \text{ bytes (tamaño de FP32)} \\ \hline \approx 40 \text{ MB (solo para almacenar pesos)} \end{array}$$

¿Necesitamos realmente 32 bits para cada peso durante la predicción?

Cuantificación: Comprimiendo el Modelo

Representar los valores del modelo utilizando una precisión numérica menor sin alterar radicalmente el comportamiento aprendido.



4 bytes por parámetro
Ej. 100 MB

Escalamiento
→



16 bits (Mitad de tamaño)
Ej. 50 MB

Escalamiento
→



Enteros de 8 bits (1 byte)
Ej. 25 MB

Mecanismo: Valores continuos (FP32) → Escalamiento → Valores discretos y limitados (INT8)

Estrategias de Cuantificación: Post-Training vs QAT

Post-Training Quantization



- Se aplica al modelo ya entrenado.
- Baja complejidad, conversión rápida.
- Ideal si ya disponemos de un modelo definitivo.

Quantization Aware Training (QAT)



- Se aplica durante el entrenamiento o reentrenamiento.
- Mayor complejidad.
- El modelo aprende a tolerar la representación, reteniendo mejor el Accuracy.

Nota: Para el proyecto integrador, la estrategia Post-Training resulta suficiente como experiencia de optimización inicial.

Implementación Práctica: Cuantificación en TensorFlow Lite

Flujo conceptual para cuantificar un modelo Keras mediante Post-Training Quantization.

CODE: Post-Training Quantization

```
converter = tf.lite.TFLiteConverter.from_keras_model(modelo)
converter.optimizations = [tf.lite.Optimize.DEFAULT]

modelo_cuantificado = converter.convert()
```

Metodología Correcta

1. Medir Accuracy / Latencia Original



2. Cuantificar Modelo (Código)



3. Medir Nuevamente



4. Comparar Resultados

Paso Clave: Una mala práctica es asumir que el modelo funciona igual tras convertirlo. **Siempre reevaluar.**

Segunda Técnica: Poda

Poda de Redes (Pruning)

Identificar y eliminar parámetros que aportan poco al comportamiento de la red, fijando su valor en 0.

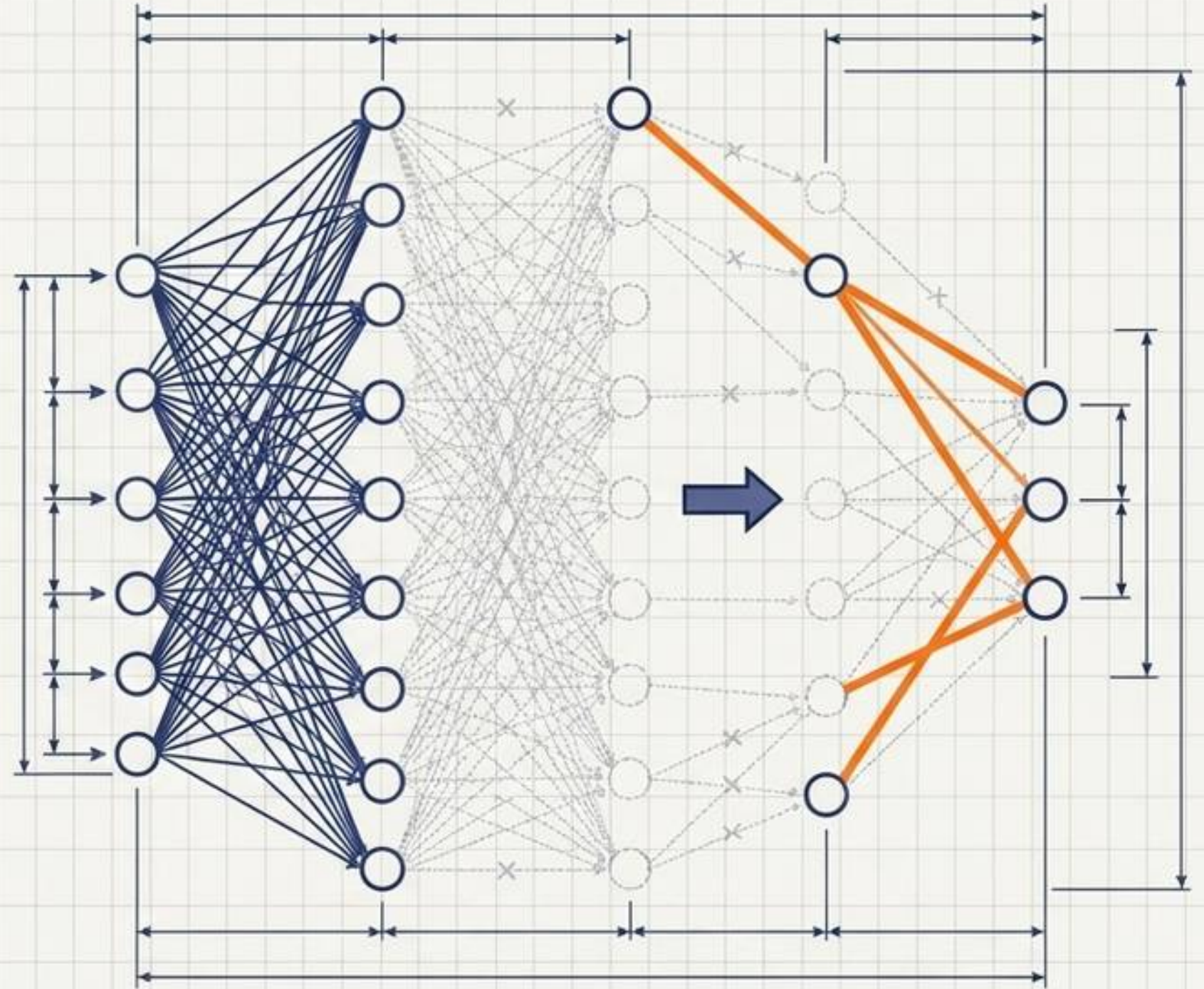
Concepto Clave: Magnitude Pruning

Si un peso es muy cercano a 0, su influencia es mínima → Se elimina la conexión efectiva.

Métrica: Sparsity (Dispersión)

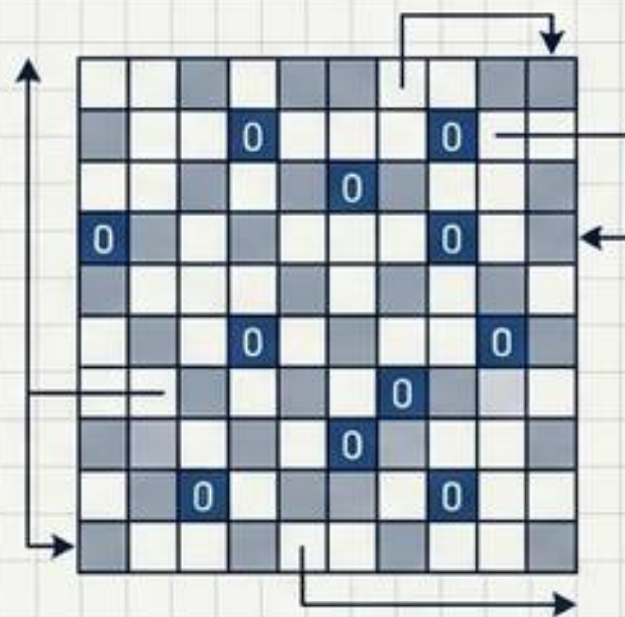
Proporción de pesos que se encuentran en cero.

$$\frac{500.000 \text{ pesos en cero}}{1.000.000 \text{ pesos totales}} = \text{50\% Sparsity}$$



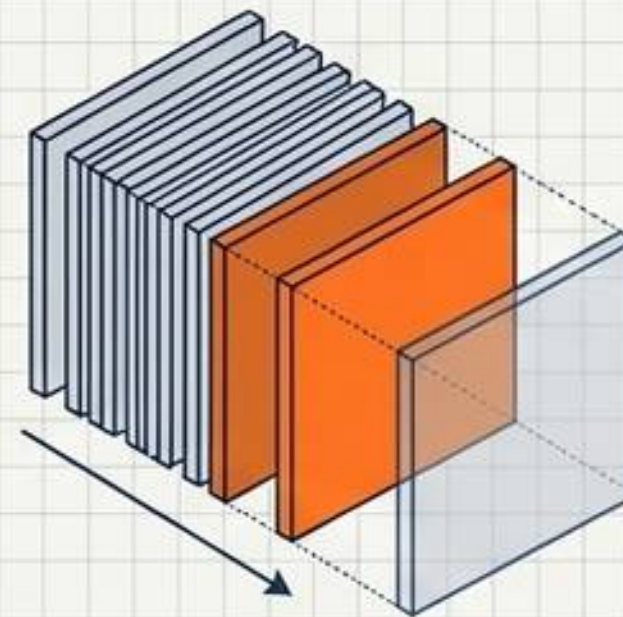
Arquitecturas Dispersas: Tipos de Poda y Fine-Tuning

Poda No Estructurada



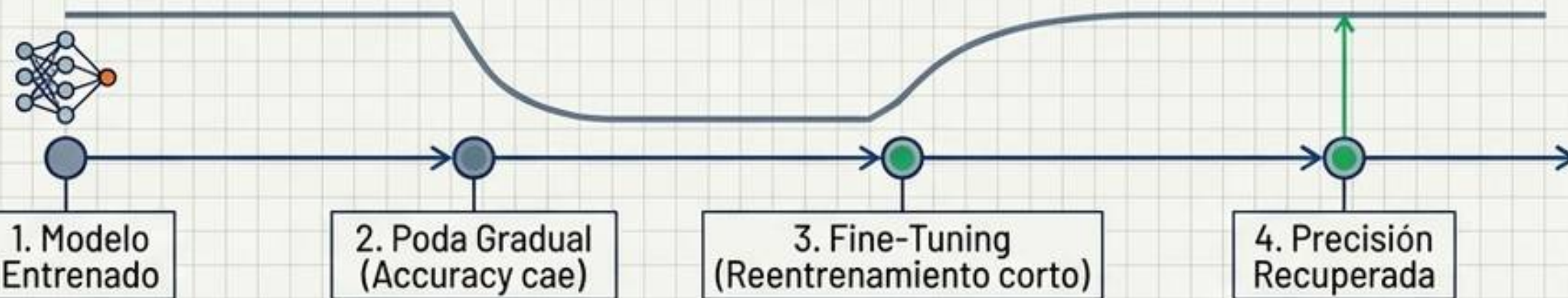
Elimina pesos individuales. La matriz conserva su tamaño pero con muchos ceros. Alta flexibilidad, difícil de acelerar en hardware.

Poda Estructurada



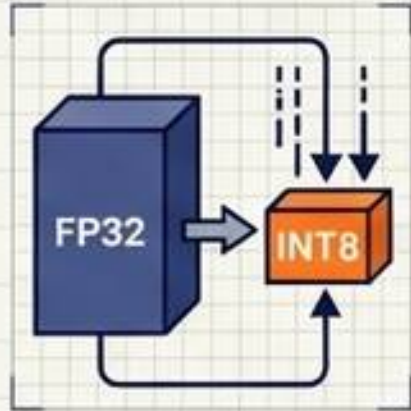
Elimina arquitecturas completas (filtros enteros, canales, neuronas). Produce una red genuinamente más pequeña y fácil de acelerar.

Poda Gradual + Fine-Tuning



Eliminar conexiones abruptamente daña el modelo. Se debe aumentar el Sparsity progresivamente y permitir que la red se adapte.

Comparativa de Transformaciones: Cuantificación vs Poda



Cuantificación (Quantization)

- **Objetivo:** Reduce la precisión de cada parámetro.
- **Mecanismo:** Menos bits por peso.
- **Ejemplo:** FP32 → INT8



Poda (Pruning)

- **Objetivo:** Reduce la cantidad de parámetros efectivos.
- **Mecanismo:** Menos conexiones útiles.
- **Ejemplo:** Peso de baja magnitud → 0

Sinergia (Optimización Acumulativa)

Ambas técnicas actúan sobre dimensiones distintas y no son excluyentes. Pueden combinarse en un mismo flujo:



Regla de Oro: Cada transformación debe medirse de manera independiente.

Metodología de Optimización: Medir con Evidencia

1. Establecer un Baseline

Toda optimización exige una línea base del modelo original. Sin ella, no hay forma objetiva de evaluar el impacto de la transformación.

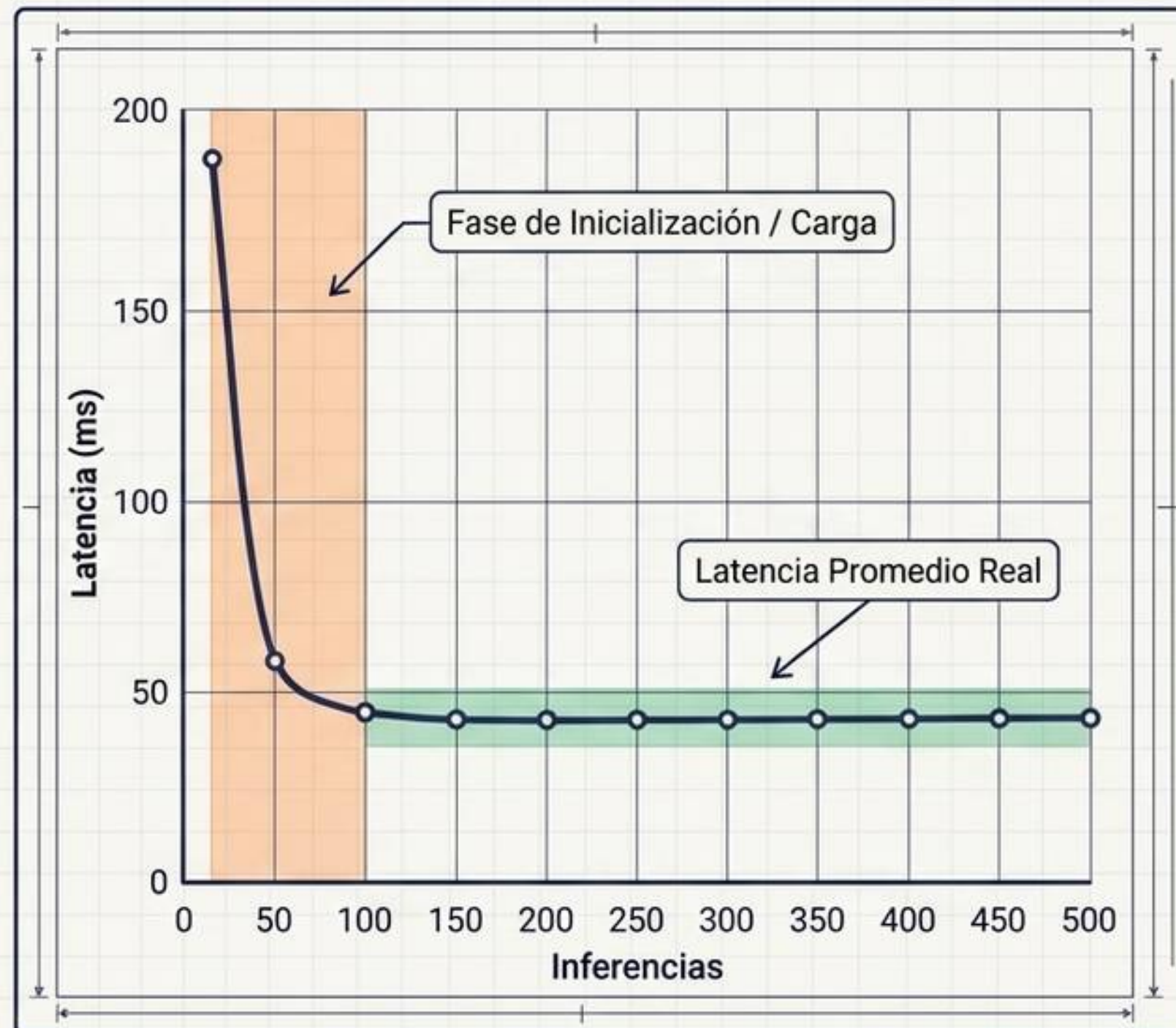
2. Medición de Latencia y Warm-up

Una sola inferencia es engañosa (incluye carga en memoria). Flujo correcto:



3. Contexto de Ejecución

- 🖼️ • **Latencia** (1 imagen): Apps interactivas.
- 🖼️ • **Throughput** (Batch): Procesamiento masivo.



El Trade-off en Acción: Clasificador de Residuos



Escenario Base (Modelo Original CNN)

- **Accuracy:** 91.5%
- **Tamaño:** 42 MB
- **Latencia:** 180 ms

Tras Cuantificación (INT8)

- **Accuracy:** 90.9% (Caída de 0.6%)
- **Tamaño:** 12 MB (Reducción del 71%)
- **Latencia:** 80 ms (Mejora del 55%)

La Decisión de Ingeniería: ¿Es aceptable perder 0.6% de precisión a cambio de reducir el peso a un tercio y duplicar la velocidad? Para un smartphone o Edge device con batería y memoria limitadas, esta es una decisión técnica sólida.

Reglas de Oro en Producción y Anti-Patrones

Trazabilidad y Versionado

Versión	Cambio	Accuracy	Tamaño
V1	CNN original	86.0%	40 MB
V2	Fine-Tuning	91.0%	40 MB
V3	Cuantificación	90.5%	12 MB

Nunca sobrescribir el archivo original.
Mantener trazabilidad.

Verificación y Anti-Patrones



Verificación Funcional

- ✓ El modelo optimizado debe aceptar el mismo preprocesamiento de Dataset.
- ✓ Debe entregar la misma cantidad de clases funcionales.
- ✓ La misión no cambia, solo su ejecución.



Anti-Patrones a Evitar

- ✗ Optimizar sin medir el Baseline inicial.
- ✗ Reportar únicamente el nuevo tamaño ignorando si el Accuracy colapsó.
- ✗ Aplicar Pruning y Cuantificación simultáneamente sin medir el impacto individual.

Síntesis: Aprendizajes de la Semana 14

1

1. Producción exige nuevas prioridades

Un modelo funcional no es implementable sin evaluar  Latencia,  Throughput,  Tamaño y Memoria.



2

2. Optimización es un Trade-off

Buscamos la mejor relación entre viabilidad de hardware y desempeño predictivo.



3

3. Cuantificación comprime parámetros

Mapa valores continuos a menor precisión (FP32 → INT8), reduciendo peso a cambio de mínima precisión.



4

4. Pruning elimina redundancias

Descarta pesos irrelevantes (aumentando Sparsity), siendo más efectivo bajo podas estructurales.



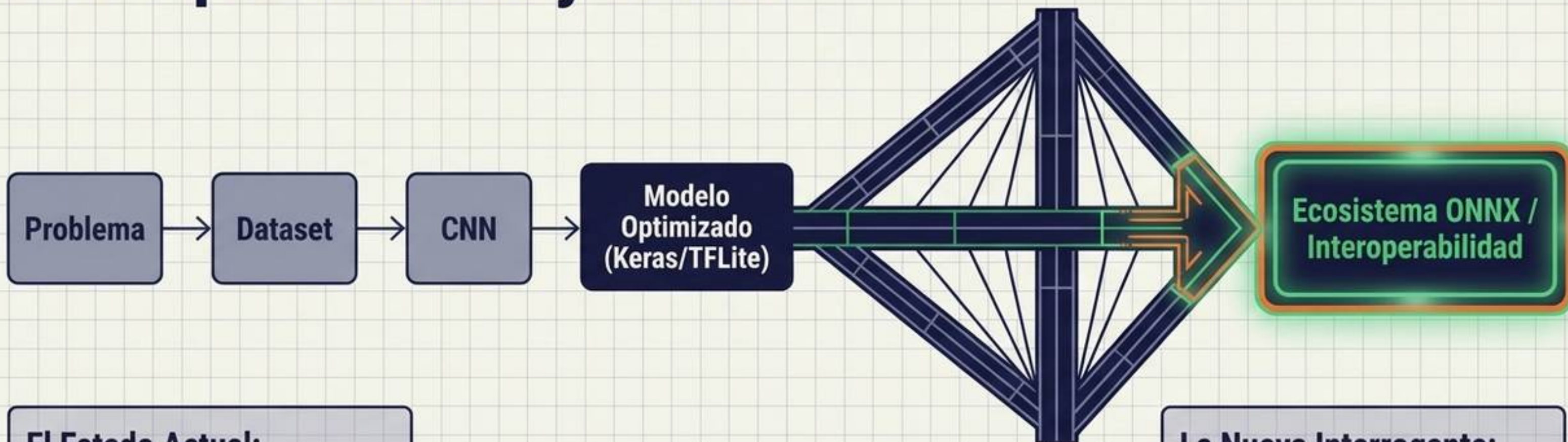
5

5. Decisiones basadas en evidencia

Toda transformación requiere Baseline previo, medición con Warm-up, y validación funcional posterior.



Hacia la Semana 15: Interoperabilidad y ONNX



El Estado Actual:

Hemos logrado transformar un modelo pesado en un artefacto altamente optimizado.

Siguiente Sesión: Exportar modelos como un formato universal para su integración en cualquier ecosistema de producción.

La Nueva Interrogante:

¿Cómo trasladamos este modelo para ser ejecutado en lenguajes como C++, C#, entornos móviles o servidores web independientemente de su framework original?